

Learning Regular Expressions

James Lee

james@onsight.com

© 2025 by Onsight

This course is for you because...

- You need tools to locate, parse, and replace text
- You want to be able to search and replace text to solve day-to-day problems, from simple to complex
- You want to learn a text processing tool that is used in many programming languages (you don't need to be a programmer or an engineer to take this course, although we will talk about code)

You Will Understand

- What regular expressions are and why they are used
- Regular expression syntax and rules
- How to read and write regular expressions
- How regular expressions are processed

You Will Be Able To

- Use Unix tools (grep, egrep, sed, awk) that use regular expressions
- Read, and more importantly, write regular expressions
- Test and validate regular expressions using online tools
- Use regular expressions in programming languages (Javascript and Python)

You Should Know and Have

- Have access to the course scenarios or an operating system
- Know basics of the shell (Linux, OS X, or Windows)
- Know basic shell commands

I Will Work in Linux

- I will be working in Linux in the class Scenarios
 - Ubuntu - <https://ubuntu.com>
 - It has tools to work with regexes (grep, sed, etc)
 - It is free



Quick Demo

- find text in a file

```
$ grep express /usr/share/dict/words  
express  
express's  
expressed  
expresses  
expressible  
expressing  
expression  
expression's  
expressionism  
expressionism's  
expressionist  
expressionist's  
expressionists  
expressionless  
expressions
```

Quick Demo

- find a file that includes text

```
$ find /usr/share/doc/mawk -exec grep -H regular {} \; 2>/dev/null
/usr/share/doc/mawk/examples/gdecl.awk:#   input : string s -- treated as a regular expression
/usr/share/doc/mawk/examples/decl.awk:#   regular expressions.  If not run on mawk, use gdecl.awk
/usr/share/doc/mawk/examples/decl.awk:#   input : string s -- treated as a regular expression
$
```

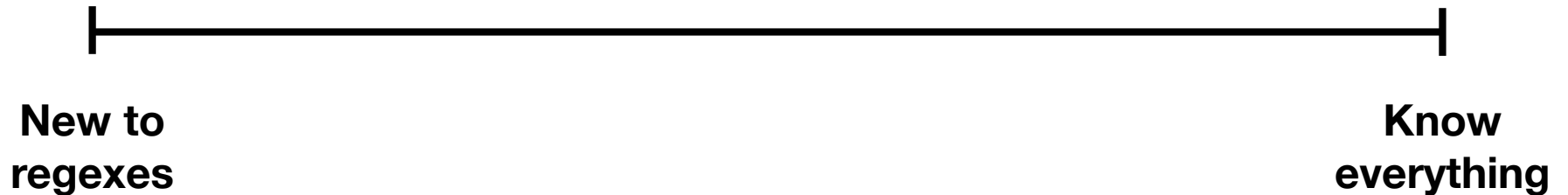

Quick Demo

- change the contents of a file

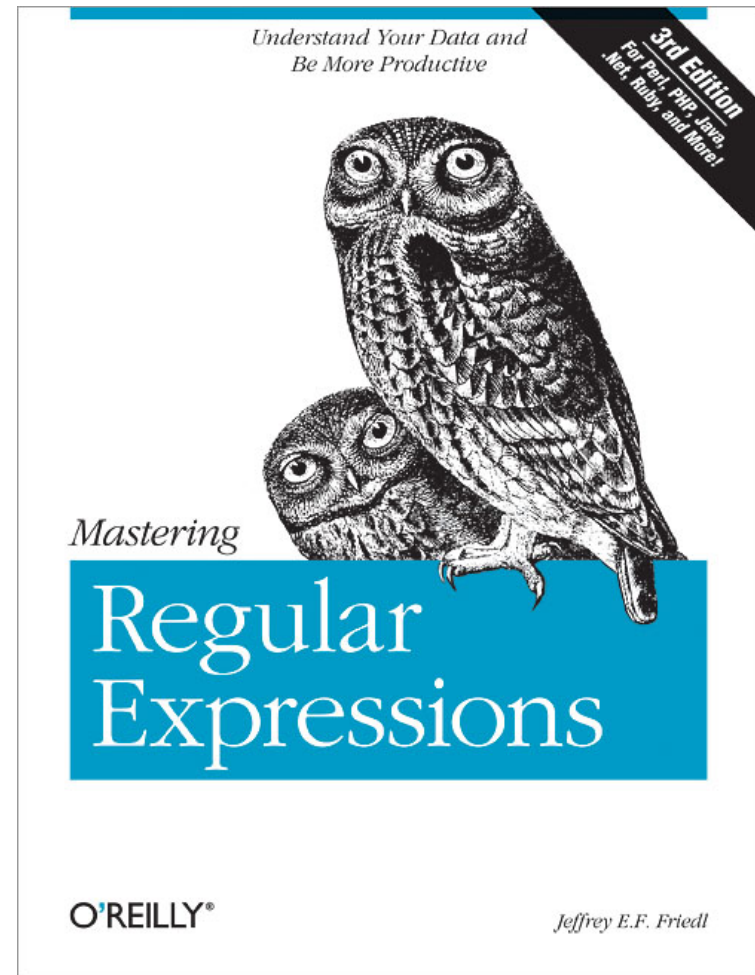
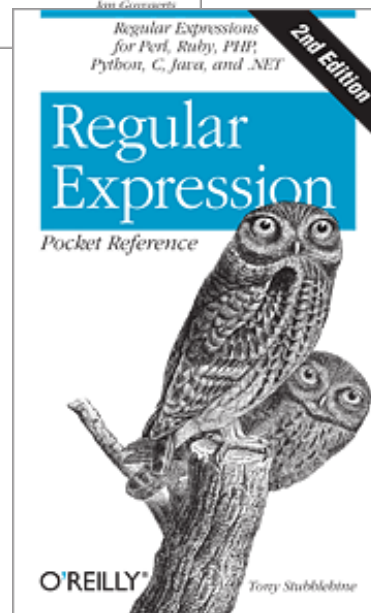
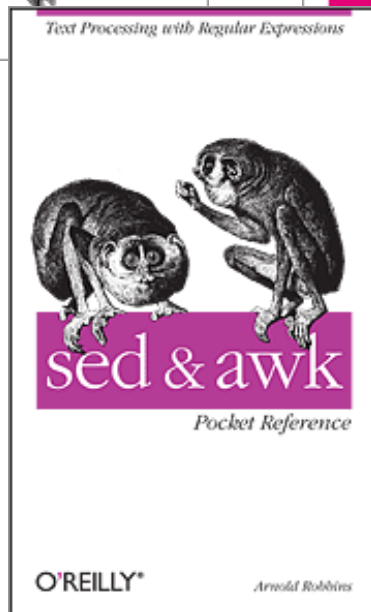
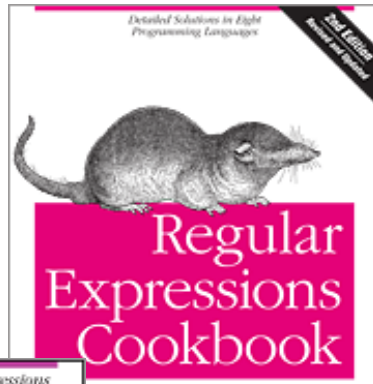
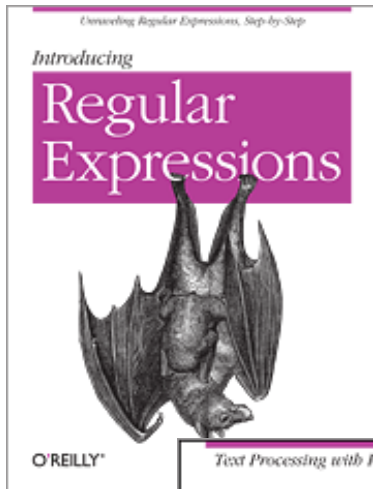
```
$ cat words.txt
zero
one
two
three
four
five
six
seven
eight
nine
$ sed -i 's/zero/ZERO/' words.txt
$ cat words.txt
ZERO
one
two
three
four
five
six
seven
eight
nine
$ sed -i 's/ZERO/zero/' words.txt
```

Range of Students

- Regexes can be used with less than advanced knowledge
- Students of regexes continue to learn more



Great Books



About This Class

- Format is: talk / exercise / break
- Exercise solutions are at the end of this PDF

Introduction to Regular Expressions

Introduction Topics

- What are regular expressions?
- Why are regular expressions important?
- Intro to `grep/egrep`
- `grep` examples
- Online tools
- Q & A

What are Regular Expressions

- also known as *regexes*
- a sequence of characters describing a *search pattern*
- used to search text
 - match text - it either matches or it doesn't

`/some regex/`

- substitute text - the matched text is replaced

`s/this/that/`

Why are Regular Expressions Important

- powerful text processing tool
 - can search for text in files

```
grep Hello file.txt
```


Why are Regular Expressions Important

- powerful text processing tool
- can search for text in files

```
grep Hello file.txt  
grep Hello *
```

Why are Regular Expressions Important

- powerful text processing tool
 - can search for text in files

```
grep Hello file.txt  
grep Hello *
```

- can search and replace text in a file

```
sed -i 's/hello/goodbye/g' msg.txt
```

Why are Regular Expressions Important

- powerful text processing tool
 - can search and replace text in a directory of files

```
sed -i 's/hello/goodbye/g' ~/dir/*
```

Why are Regular Expressions Important

- can be used by many Linux utilities
 - `grep`, `sed`, `awk`, `find`

Yeah, but Regular Expressions are Complicated!

```
/(?x) , [ ^ , ] ++ , /
```

```
/(?<=\.) {2,} (?=[A-Z])/
```

```
/\(?\d{3}[\ -.\)]*\d{3}[\ -.]? \d{4}/
```

```
/(?x) (? : https? | ftp) :// ([^/]+) (/.*)? /
```

```
s/(?<=\d)(?=(\d\d\d)+(?!\d))/,/g
```

```
/(?xi) ^ [\w+-]+ (? : \. [\w.+-]+ ) * @ [a-z0-9-]+ (? : \. [a-z0-9-]+ ) * $ /
```

Yeah, but Regular Expressions are Complicated!

- powerful tools are complex
- with regexes, we can do a lot with just a little knowledge
- the more we know, the more we can do



- regexes are a language, and like other languages, you can say a lot with a little
- the purpose of this class is to learn enough so the complicated becomes understandable

Regex Flavors

- 6 major dialects of regexes
 - BRE - Base Regular Expressions (`grep`)
 - ERE - Extended Regular Expressions (`egrep`)
 - Emacs
 - ViM
 - PCRE - Perl Compatible Regular Expressions (Python, Javascript) (`grep -P`) - this is what we will use
 - Perl6

Regular Expressions are Another Language

- roots are in mathematics, computer science theory
- regexes are code (with "statements", "loops" and "subroutines")
 - can apply code design skills
- regexes run in a specialized virtual machine
 - finite state machine - deterministic and non-deterministic

Intro to `grep` / `egrep`

- in `ed`, to print all lines that match a regex:

```
g/re/p
```

Intro to grep / egrep

- grep uses the BRE dialect
- egrep uses the ERE dialect
 - egrep is equivalent to grep -E
- grep -P (in Ubuntu) uses PCRE - we will use PCRE in this class
 - it has the most features
 - it is supported by Javascript and Python
- <https://remram44.github.io/regex-cheatsheet/regex.html>

grep/egrep Examples

- the first argument to grep is a regular expression, the second argument is the file

```
$ grep hello /usr/share/dict/words
0thello
0thello's
hello
hello's
hello's
$
```

grep/egrep Examples

- with grep, sometimes it is necessary to quote the regex, and usually ok to do so even if it not necessary:

```
$ grep 'hello' /usr/share/dict/words
Othello
Othello's
hello
hello's
hellos
$
```

grep/egrep Examples

- difference between grep and egrep:

```
$ grep 'hello|goodbye' /usr/share/dict/words
$ grep 'hello\\|goodbye' /usr/share/dict/words
0thello
0thello's
goodbye
goodbye's
goodbyes
hello
hello's
hellos
$ egrep 'hello|goodbye' /usr/share/dict/words
0thello
0thello's
goodbye
goodbye's
goodbyes
hello
hello's
hellos
$
```

grep/egrep Examples

- case insensitive match with the `-i` option:

```
$ grep austr /usr/share/dict/words
claustrophobia
claustrophobia's
claustrophobic
$ grep -i austr /usr/share/dict/words
Australasia
Australasia's
Australia
Australia's
Australian
Australian's
Australians
Australoid
Australoid's
Australopithecus
Australopithecus's
Austria
Austria's
Austrian
Austrian's
Austrians
Austronesian
Austronesian's
claustrophobia
claustrophobia's
claustrophobic
```

grep/egrep Examples

- read regexes from a file with the `-f` option:

```
$ cat regexes.txt
Macb
Austra
Illi
$ grep -f regexes.txt /usr/share/dict/words
Australasia
Australasia's
Australia
Australia's
Australian
Australian's
Australians
Australoid
Australoid's
Australopithecus
Australopithecus's
Illinois
Illinois's
Macbeth
Macbeth's
```

grep/egrep Examples

- show all lines the don't match with the `-v` option:

```
$ cat words.txt
zero
one
two
three
four
five
six
seven
eight
nine
$ grep ne words.txt
one
nine
$ grep -v ne words.txt
zero
two
three
four
five
six
seven
eight
```


grep for Windows

- grep is installed in Linux and OS X
- grep is available for Windows
- <http://gnuwin32.sourceforge.net/packages/grep.htm>

PCRE for MacOS

- grep in MacOS, does not support PCRE
- grep with PCRE is available:

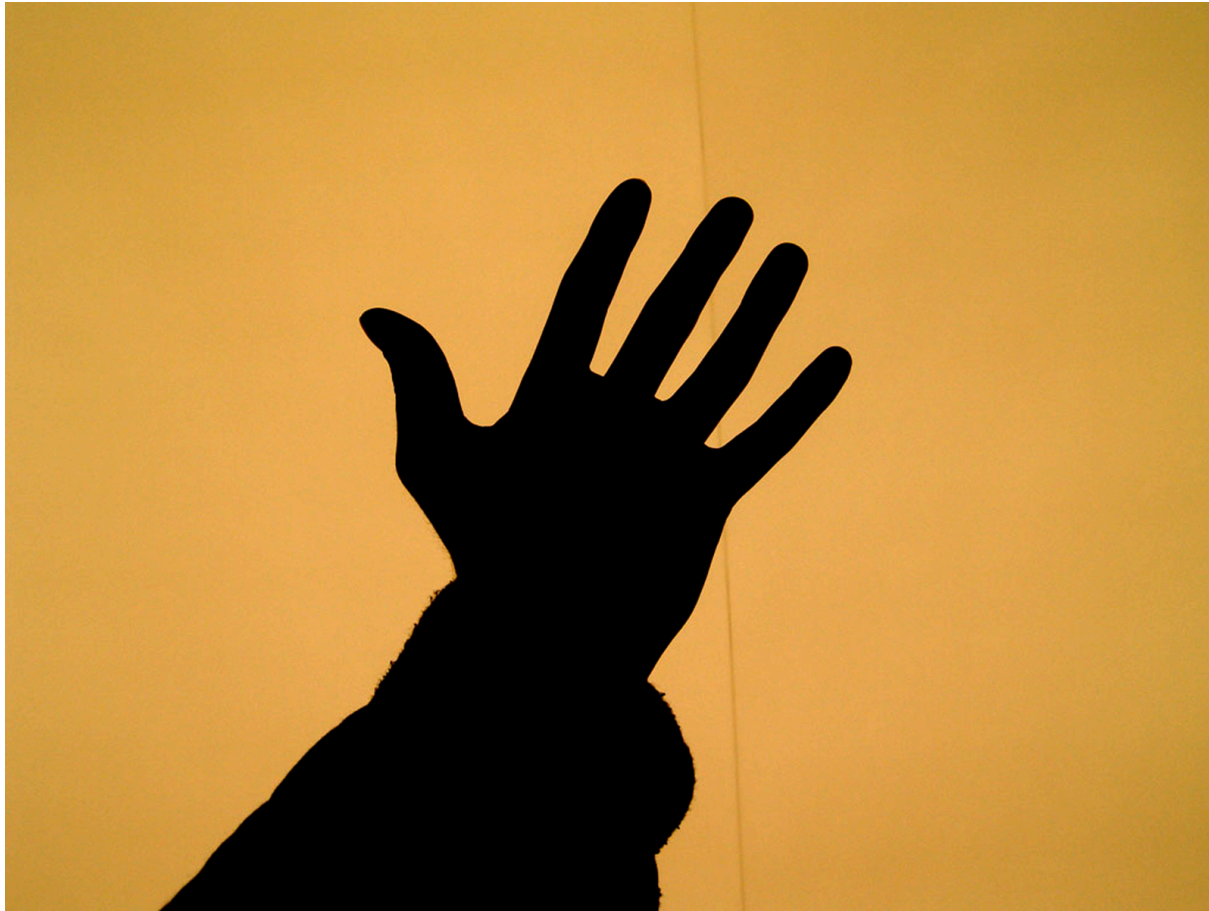
```
brew update  
brew install grep
```

The improved grep is called ggrep

Online Tools

- <https://regexr.com/>
- <https://regex101.com/>
- <https://www.regextester.com/>
- <https://www.freeformatter.com/regex-tester.html>
- <https://regexcrossword.com/>

Q & A



Basic Syntax

Basic Syntax Topics

- Basic regular expressions - normal characters and .
- Beginning and ending of the line - ^ and \$
- Regex engine basics
- Regular Expression Rule #1
- Exercise: Basic Syntax
- Exercise solution and Q & A

Basic Regexes

- Most characters (alpha-numeric) match themselves:

```
grep a file.txt
```

match "a" anywhere in the string (line of the file)

Basic Regexes

- Most characters (alpha-numeric) match themselves:

```
grep a file.txt
```

match "a" anywhere in the string (line of the file)

```
grep abc file.txt
```

match "a" followed by "b" followed by "c" anywhere in the string

Basic Regexes

- Most characters (alpha-numeric) match themselves:

```
grep a file.txt
```

match "a" anywhere in the string (line of the file)

```
grep abc file.txt
```

match "a" followed by "b" followed by "c" anywhere in the string

```
grep ab12 file.txt
```

match "a" followed by "b" followed by "1" followed by "2" anywhere in the string

Basic Regexes

- Many characters have special meaning
 - . - matches any character except \n:

```
grep a.b file.txt
```

match "a" followed by any character but \n followed by "b", anywhere in the string

Basic Regexes

- Many characters have special meaning

. - matches any character except \n:

```
grep a.b file.txt
```

match "a" followed by any character but \n followed by "b", anywhere in the string

```
grep a.b.c file.txt
```

match "a" followed by any character but \n followed by "b" followed by any character but \n followed by "c", anywhere in the string

Begin and End of String

- We can match the beginning or end of the string:
 - ^ - match the beginning of the string
 - \$ - match the end of the string (or right before newline at the end of the string)

Begin and End of String

```
grep ^abc file.txt
```

match "abc" at the beginning of the string

Begin and End of String

```
grep ^abc file.txt
```

match "abc" at the beginning of the string

```
grep abc$ file.txt
```

match "abc" at the end of the string (or "abc\n" at end of string)

Begin and End of String

```
grep ^abc file.txt
```

match "abc" at the beginning of the string

```
grep abc$ file.txt
```

match "abc" at the end of the string (or "abc\n" at end of string)

```
grep ^abc$ file.txt
```

match the string "abc" (also matches "abc\n")

To Match Special Characters

- To match special characters, they must be escaped with the backslash:

```
grep a\.b\.c file.txt
```

```
grep ^\^\$\$ file.txt
```


Basic Regexes Example

- In the home directory is a file `words.txt`:

```
zero  
one  
two  
three  
four  
five  
six  
seven  
eight  
nine
```

Basic Regexes Example

```
$ grep -P n words.txt
one
seven
nine
$ grep -P ne words.txt
one
nine
$ grep -P r words.txt
zero
three
four
$ grep -P ^n words.txt
nine
$ grep -P r$ words.txt
four
$ grep -P y words.txt
$
```

Regex "Statements"

- Think of chars in a regex as statements:

`/^abc$ /`

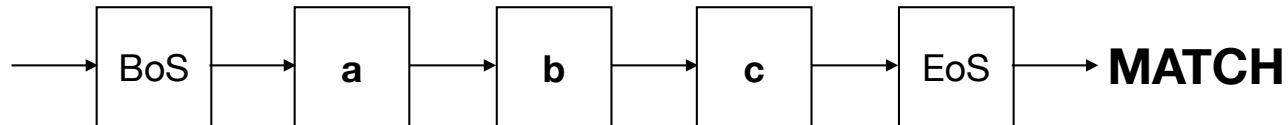
- if "a" at the beginning of the string
- and then if "b"
- and then if "c" at the end of the string
- then MATCH



Regex State Machine

- Can think of the regex as a state machine:

`/^abc$/`



Regex Engine Basics

- start at left of regex
- start at left of string
- try all possible paths
- backtrack as little as possible (more on this later)
- if MATCH, stop - SUCCESS 
- if FAIL, move along one character in string, start over at the beginning of the regex, and try all possible paths again
- if FAIL starting at every character - FAIL 

Regex Rule #1

- The earliest match wins:

"a fundamental idea of regexes: they are fun"

`/fun/`

Regex Rule #1

- The earliest match wins:

"a **fun**damental idea of regexes: they are fun"

/fun/

Regex Engine

"abcdefg"

/abc/

Regex Engine

"abcdefg"
↑

/abc/
↑

Regex Engine

"a**a**bcdefg"



/abc/



Regex Engine

"**ab**cdefg"



/abc/



Regex Engine

"abcdefg"
↑

/abc/
↑

MATCH

Regex Engine

"ababcdefg"

/abc/

Regex Engine

"ababcdefg"



/abc/



Regex Engine

"ababcfg"



/abc/



Regex Engine

"ab^{ab}abcdefg"



/abc/



Regex Engine

"ababcdefg"



/abc/



Regex Engine

"ababcdefg"



/abc/



Regex Engine

"ab**a**bcdefg"



/abc/



Regex Engine

"ab**abc**defg"



/abc/



Regex Engine

"ab**abc**defg"



/abc/



MATCH

Regex Engine

"ababab"



/abc/



Regex Engine

"a**a**babab"



/abc/



Regex Engine

"ababab"



/abc/



Regex Engine

"ababab"



/abc/



Regex Engine

"ababab"



/abc/



Regex Engine

"ababab"



/abc/



Regex Engine

"ab**ab**ab"



/abc/



Regex Engine

"ababab"



/abc/



Regex Engine

"ababab"



/abc/



Regex Engine

"ababab"



/abc/



Regex Engine

"ababab"
↑

/abc/
↑

FAIL

Exercise 1

- In the home directory is a file `words.txt`:

zero
one
two
three
four
five
six
seven
eight
nine

- Which words match the following:

1. `t`
2. `.n.`
3. `^t`
4. `e$`
5. `ne$`
6. `^...$`

Exercise 1

- Write regexes to do the following:
 - match lines with e anywhere in the line
 - match lines that start with e
 - match 4 character lines that begin with £ and end with e

Q & A



Character Classes

Character Classes Topics

- Character class syntax - in a class
- Character class syntax - not in a class
- POSIX classes
- Predefined classes
- Exercise: Character classes
- Exercise solution and Q & A

In a Class

- A character class matches one character, any character in the class:

`/[abcde]/`

match one character, either a, b, c, d or e

In a Class

- A character class matches one character, any character in the class:

`/[abcde]/`

match one character, either a, b, c, d or e

`/[a-e]/`

same - match one character a through e

In a Class

- A character class matches one character, any character in the class:

`/[abcde]/`

match one character, either a, b, c, d or e

`/[a-e]/`

same - match one character a through e

`/[a-z]/`

match one lower case alpha

In a Class

- A character class matches one character, any character in the class:

`/[abcde]/`

match one character, either a, b, c, d or e

`/[a-e]/`

same - match one character a through e

`/[a-z]/`

match one lower case alpha

`/[0-9]/`

match one digit character

Character Class Example 1

- In the home directory is a file `text.txt`:

`testing`

`test1`

`test123`

`1-one`

`2-two`

`one-1`

`two-2`

Character Class Example 1

```
$ grep -P [a-z] text.txt
testing
test1
test123
1-one
2-two
one-1
two-2
$ grep -P [a-z][0-9] text.txt
test1
test123
$ grep -P [a-z-][0-9] text.txt
test1
test123
one-1
two-2
$ grep -P [a-z\-][0-9] text.txt
test1
test123
one-1
two-2
$
```

Character Class Example 1

- In the home directory is a file `phonenumbers.txt`:

3125551212

312-555-1212

312 555-1212

312 555 1212

(312) 555-1212

312.555.1212

Character Class Example 1

```
$ grep -P [0-9][0-9][0-9][0-9] phonenumbers.txt
3125551212
312-555-1212
312 555-1212
312 555 1212
(312) 555-1212
312.555.1212
$ grep -P '[0-9][0-9][0-9][ \-][0-9][0-9][0-9]' phonenumbers.txt
312-555-1212
312 555-1212
312 555 1212
(312) 555-1212
$ _
```

Not In a Class

- If the caret is the first character in a class, it means match a character not in the class

`/[^abcde]/`

match one character, neither a, b, c, d nor e

Not In a Class

- If the caret is the first character in a class, it means match a character not in the class

`/[^abcde]/`

match one character, neither a, b, c, d nor e

`/[^a-e]/`

same - match one character, not a through e

Not In a Class

- If the caret is the first character in a class, it means match a character not in the class

`/[^abcde]/`

match one character, neither a, b, c, d nor e

`/[^a-e]/`

same - match one character, not a through e

`/[^a-z]/`

match one non lower case alpha

Not In a Class

- If the caret is the first character in a class, it means match a character not in the class

`/[ ^abcde ]/`

match one character, neither a, b, c, d nor e

`/[ ^a-e ]/`

same - match one character, not a through e

`/[ ^a-z ]/`

match one non lower case alpha

`/[ ^0-9 ]/`

match one non digit character

Character Class Example 2

```
$ grep -P [^a-z] text.txt
test1
test123
1-one
2-two
one-1
two-2
$ grep -P [^a-z][^0-9] text.txt
1-one
2-two
$ grep -P [^0-9][^a-z] text.txt
testt1
testt123
onee-1
twoo-2
$
```

POSIX Character Classes

- Some groups are very common, for instance:

`/[a-z]/`

- This class can be replaced with one of the POSIX character classes (don't forget the outer `[ ]`):

`/[:lower:]/`

POSIX Character Classes

<code>alnum</code>	- letters and digits <code>[a-zA-Z0-9]</code>
<code>alpha</code>	- letters <code>[a-zA-Z]</code>
<code>ascii</code>	- ascii codes 0 - 127
<code>blank</code>	- space or tab <code>[\t]</code>
<code>cntrl</code>	- control characters
<code>digit</code>	- digits <code>[0-9]</code>
<code>graph</code>	- printing characters, excluding space
<code>lower</code>	- lower case letters <code>[a-z]</code>
<code>print</code>	- printing characters, including space
<code>punct</code>	- printing chars, excl letters, digits, space
<code>space</code>	- white space <code>[\t\n\f\r]</code> and VT
<code>upper</code>	- upper case letters <code>[A-Z]</code>
<code>word</code>	- word characters <code>[a-zA-Z0-9_]</code>
<code>xdigit</code>	- hex digits

Negate POSIX Character Classes

- To negate, include the caret after the first colon:

```
[[:^lower:]]
```

```
[[:^digit:]]
```

Negate POSIX Character Classes

- To negate, include the caret after the first colon:

```
[[:^lower:]]  
[[:^digit:]]
```

- Examples:

```
/[12[:^digit:]]/
```

match one char, either 1, 2 or non-digit

Negate POSIX Character Classes

- To negate, include the caret after the first colon:

```
[[:^lower:]]  
[[:^digit:]]
```

- Examples:

```
/[12[:^digit:]]/
```

match one char, either 1, 2 or non-digit

```
/[abc[:^lower:]]/
```

match one char, either a, b, c or non-lower

Character Class Example 3

```
$ grep -P [[:lower:]] text.txt
testing
test1
test123
1-one
2-two
one-1
two-2
$ grep -P [[:digit:]] text.txt
test1
test123
1-one
2-two
one-1
two-2
$ grep -P [[:^lower:]] text.txt
test1
test123
1-one
2-two
one-1
two-2
$
```


Generic Character Classes

- Some character classes are so common there is a shorthand version:

```
\d    - digit [0-9]
\D    - non-digit [^0-9]
\w    - word [a-zA-Z0-9_]
\W    - non-word [^a-zA-Z0-9_]
\s    - space character [ \t\n\r\f]
\S    - non-space [^ \t\n\r\f]
\h    - horizontal white space
\H    - non-horizontal white space
\v    - vertical white space
\V    - non-vertical white space
```

Character Class Example 4

```
$ grep -P '\d\d\d\d' phonenumbers.txt
3125551212
312-555-1212
312 555-1212
312 555 1212
(312) 555-1212
312.555.1212
$ grep -P '\d\d\d-\d\d\d\d' phonenumbers.txt
312-555-1212
312 555-1212
(312) 555-1212
$ grep -P '\d\d\d[ \-.] \d\d\d\d' phonenumbers.txt
312-555-1212
312 555-1212
312 555 1212
(312) 555-1212
312.555.1212
$
```

Exercise 2

- In the home directory is a file `words.txt`:

zERo
One
Two
tHREE
four
FIVE
six
Seven
eight
nine
Ten 10
eleven 11

- Which words match the following:

1. `[0-9]`
2. `[A-Z][a-z]`
3. `\w\s\d`
4. `[[ :upper: ]]`
5. `^[[ :upper: ]]`
6. `[[ :^alpha: ] ]$`

Exercise 2

- Write regexes to do the following:
 - match lines in `words.txt` that have at least 3 upper alphas in a row
 - match lines in `phonenumbers.txt` that have 3 digits followed by a space, dash or period followed by 3 digits followed by a space, dash or period followed by 4 digits (use generic classes)

Q & A



Quantifiers

Quantifiers Topics

- Quantifier syntax
- Regular Expression Rule #2
- Embedding whitespace - more readable regexes
- Exercise: Quantifiers
- Exercise solution and Q & A

Quantifier Syntax

* – zero or more

ab*c – "a", zero or more "b", "c"

Quantifier Syntax

* – zero or more

ab^*c – "a", zero or more "b", "c"

+ – one or more

ab^+c – "a", one or more "b", "c"

Quantifier Syntax

* – zero or more

ab^*c – "a", zero or more "b", "c"

+ – one or more

ab^+c – "a", one or more "b", "c"

? – zero or one

$ab?c$ – "a", zero or one "b", "c"

Quantifier Syntax

* – zero or more

ab^*c – "a", zero or more "b", "c"


+ – one or more

ab^+c – "a", one or more "b", "c"

? – zero or one

$ab?c$ – "a", zero or one "b", "c"

Quantifier Syntax

* – zero or more

ab^*c – "a", zero or more "b", "c"


ac abc abbc abbbc ...

+ – one or more

ab^+c – "a", one or more "b", "c"

? – zero or one

$ab?c$ – "a", zero or one "b", "c"

Quantifier Syntax

* – zero or more

ab^*c – "a", zero or more "b", "c"


ac abc abbc abbbc ...

+ – one or more

ab^+c – "a", one or more "b", "c"


? – zero or one

$ab?c$ – "a", zero or one "b", "c"

Quantifier Syntax

* – zero or more

ab^*c – "a", zero or more "b", "c"


ac abc abbc abbbc ...

+ – one or more

ab^+c – "a", one or more "b", "c"


abc abbc abbbc ...

? – zero or one

$ab?c$ – "a", zero or one "b", "c"

Quantifier Syntax

* – zero or more

ab^*c – "a", zero or more "b", "c"


ac abc abbc abbbc ...

+ – one or more

ab^+c – "a", one or more "b", "c"


abc abbc abbbc ...

? – zero or one

$ab?c$ – "a", zero or one "b", "c"


Quantifier Syntax

* – zero or more

ab^*c – "a", zero or more "b", "c"


ac abc abbc abbbc ...

+ – one or more

ab^+c – "a", one or more "b", "c"


abc abbc abbbc ...

? – zero or one

$ab?c$ – "a", zero or one "b", "c"


ac abc

Quantifier Syntax

$\{n\}$ – n times

$ab\{3\}c$ – "a", 3 "b", "c"

$\{n, \}$ – n or more

$ab\{3, \}c$ – "a", three or more "b", "c"

$\{n, m\}$ – n through m

$ab\{3, 5\}c$ – "a", 3, 4 or 5 "b", "c"

Quantifier Syntax

$\{n\}$ – n times

$ab\{3\}c$ – "a", 3 "b", "c"


$\{n, \}$ – n or more

$ab\{3, \}c$ – "a", three or more "b", "c"

$\{n, m\}$ – n through m

$ab\{3, 5\}c$ – "a", 3, 4 or 5 "b", "c"

Quantifier Syntax

$\{n\}$ – n times

$ab\{3\}c$ – "a", 3 "b", "c"



abbbbc

$\{n, \}$ – n or more

$ab\{3, \}c$ – "a", three or more "b", "c"

$\{n, m\}$ – n through m

$ab\{3, 5\}c$ – "a", 3, 4 or 5 "b", "c"

Quantifier Syntax

$\{n\}$ - n times

$ab\{3\}c$ - "a", 3 "b", "c"



abbbc

$\{n, \}$ - n or more

$ab\{3, \}c$ - "a", three or more "b", "c"



$\{n, m\}$ - n through m

$ab\{3, 5\}c$ - "a", 3, 4 or 5 "b", "c"

Quantifier Syntax

$\{n\}$ - n times

$ab\{3\}c$ - "a", 3 "b", "c"



abbbc

$\{n, \}$ - n or more

$ab\{3, \}c$ - "a", three or more "b", "c"



abbbc abbbbc abbbbbc abbbbbbc ...

$\{n, m\}$ - n through m

$ab\{3, 5\}c$ - "a", 3, 4 or 5 "b", "c"

Quantifier Syntax

$\{n\}$ - n times

$ab\{3\}c$ - "a", 3 "b", "c"


$\{n, \}$ - n or more

$ab\{3, \}c$ - "a", three or more "b", "c"


$abbbbc \ abbbbbc \ abbbbbc \ abbbbbc \ \dots$

$\{n, m\}$ - n through m

$ab\{3, 5\}c$ - "a", 3, 4 or 5 "b", "c"


Quantifier Syntax

$\{n\}$ - n times

$ab\{3\}c$ - "a", 3 "b", "c"


$\{n, \}$ - n or more

$ab\{3, \}c$ - "a", three or more "b", "c"


$abbbc \ abbbbc \ abbbbbc \ abbbbbbc \ \dots$

$\{n, m\}$ - n through m

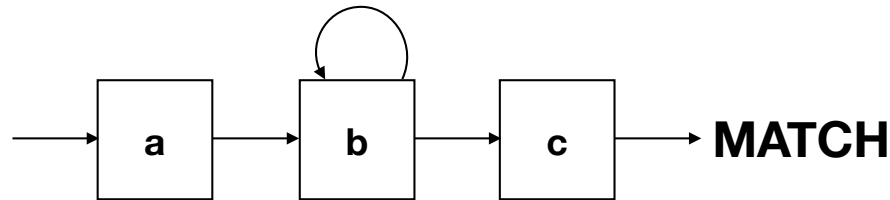
$ab\{3, 5\}c$ - "a", 3, 4 or 5 "b", "c"


$abbbc \ abbbbc \ abbbbbc$

Quantifiers Are "Loops"

- Think of quantifiers as "loops"

/ab+c/



Quantifier Example

```
$ grep -P '\w{3}\s\d{2}' words.txt
```

Ten 10

eleven 11

```
$ grep -P '[A-Z]{4}' words.txt
```

tHREE

FIVE

```
$ grep -P '[:upper:]{4}' words.txt
```

tHREE

FIVE

Quantifier Example

```
$ grep -P '\d{3}-\d{4}' phonenumbers.txt
312-555-1212
312 555-1212
(312) 555-1212
$ grep -P '\d{3}[\ -.]\d{4}' phonenumbers.txt
312-555-1212
312 555-1212
312 555 1212
(312) 555-1212
312.555.1212
$ grep -P '\d{3}[\ -.]? \d{4}' phonenumbers.txt
3125551212
312-555-1212
312 555-1212
312 555 1212
(312) 555-1212
312.555.1212
$
```

Quantifier Example

```
$ cat phonenumbers.txt
3125551212
312-555-1212
312 555-1212
312 555 1212
(312) 555-1212
312.555.1212
$ grep -P '\(?\d{3}[ \-\.\)]*\d{3}[ \-\.]?\d{4}' phonenumbers.txt
3125551212
312-555-1212
312 555-1212
312 555 1212
(312) 555-1212
312.555.1212
$
```

Regex Rule #2

- Quantifiers are greedy - they consume as much as they can

`"regexes are really not that difficult"`

`/re.*1/`

Regex Rule #2

- Quantifiers are greedy - they consume as much as they can

"`regexes are really not that difficult`"

`/re.*1/`

Regex Rule #2

- Quantifiers are greedy - they consume as much as they can

"regexes are really not that difficult"
↑

/re.*1/
↑

Regex Rule #2

- Quantifiers are greedy - they consume as much as they can

"**r**egexes are really not that difficult"

/re.*1/
↑

Regex Rule #2

- Quantifiers are greedy - they consume as much as they can

"**re**gexes are really not that difficult"

/re.*1/

Regex Rule #2

- Quantifiers are greedy - they consume as much as they can

"**re**gexes are really not that difficult"

/re.*1/

Regex Rule #2

- Quantifiers are greedy - they consume as much as they can

"**re**gexes are really not that difficult"

/re.*1/

Regex Rule #2

- Quantifiers are greedy - they consume as much as they can

"`regexes are really not that difficult`"


`/re.*1/`


Regex Rule #2

- Quantifiers are greedy - they consume as much as they can

"**re**gexes are really not that difficu**l**t"

/re.***l**/

MATCH

Regex Engine

"abccabccabcc"

/ab.*c/

Regex Engine

"abcabcabcab"



/ab.*c/



Regex Engine

"**a**bcabcbcabcb"



/ab.*****c/



Regex Engine

"**ab**cabcabcab"



/ab.*****c/



Regex Engine

"**ab**cabcabcb**c**"
↑

/ab.*****c/
↑

Regex Engine

"**ab**cab**cab**cab**c**"
↑

/ab.*****c/
↑

Regex Engine

"abcabcabc"
↑

/ab.*c/
↑

MATCH

Regex Engine

`"abcdefghijkl"`

`/ab.*c/`

Regex Engine

"abcdefghijkl"



/ab.*c/



Regex Engine

"a**a**bcdefghijkl"



/ab.**a***c/



Regex Engine

"**ab**cdefghijkl"



/ab.*****c/



Regex Engine

"abcdefghijkl"
↑

/ab.^*c/
↑

Regex Engine

"**a**bcd**e**fgh**i**jk**l**"

`/ab.*c/`

Regex Engine

"**a**bcd**e**fgh**i**j**k**l"

`/ab.*c/`

Regex Engine

"**a**bcd**e**fgh**i**jk**l**"

`/ab.*c/`

Regex Engine

"**ab**cdefghijkl"



/ab.*c/



jumping ahead (er, behind)

Regex Engine

"abcdefghijkl"
↑

/ab.*c/
↑

MATCH

(but a lot of extra work)

So...

- Quantifiers are by default *greedy* (aka *maximal*)

So...

- Quantifiers are by default *greedy* (aka *maximal*)
- They match as much as they can

So...

- Quantifiers are by default *greedy* (aka *maximal*)
- They match as much as they can
- When necessary, they *backtrack*

So...

- Quantifiers are by default *greedy* (aka *maximal*)
- They match as much as they can
- When necessary, they *backtrack*
- Backtracking can be expensive

So...

- Quantifiers are by default *greedy* (aka *maximal*)
- They match as much as they can
- When necessary, they *backtrack*
- Backtracking can be expensive
- There is a way to match the opposite: *lazy* (aka *minimal*)
(more on this later)

So...

- Quantifiers are by default *greedy* (aka *maximal*)
- They match as much as they can
- When necessary, they *backtrack*
- Backtracking can be expensive
- There is a way to match the opposite: *lazy* (aka *minimal*)
(more on this later)
- There is a way to turn off backtracking (more on this later)

Embedding Whitespace

- Before things get more complicated...

Embedding Whitespace

- Before things get more complicated...
- One way to make regexes more readable...

Embedding Whitespace

- Before things get more complicated...
- One way to make regexes more readable...
- Is to embed whitespace with:

(?x)

Embedding Whitespace

```
/(?x) \(? \d{3} [ \-.\)]* \d{3} [ \-.]? \d{4}/
```

```
/(?x)  
  \(?          # an optional open paren  
  \d{3}        # area code  
  [ \s\-\.)]* # any number of separator chars  
  \d{3}        # prefix  
  [ \s\-\.]?   # optional separator  
  \d{4}        # line number  
/
```

Embedding Whitespace

- To include the space character:

```
/(?x) hello \x20 world /
```


Embedding Whitespace Example

```
$ grep -P '(?x) \d{3} [ \-.\)]* \d{3} [ \-.\)]? \d{4}' phonenumbers.txt
3125551212
312-555-1212
312 555-1212
312 555 1212
(312) 555-1212
312.555.1212
$ grep -P '(?x) \d{3} \x20 \d{3} \x20 \d{4}' phonenumbers.txt
312 555 1212
```

Exercise 3

- In the home directory is a file `words.txt`:

zERo
One
Two
tHREE
fOuR
FIVE
six
Seven
eight
nine
Ten 10
eleven 11

- Which words match the following:

1. `[A-Z]{2}`
2. `[a-z][A-Z]{2}`
3. `([a-z][A-Z]){2}`
4. `[A-Z]+`
5. `[a-zA-Z]+\s\d+`
6. `\w+\s\w+`
7. `(?x) \w+ \x20? \w+`

Exercise 3

- Create regexes to match the following, embedding whitespace in each:
 1. lines that begin and end with 3 digits (use generic classes)
 2. lines that begin with 3 digits, have 2 lower case vowels in a row somewhere in the line, and end with 3 lower case characters
 3. lines that begin and end with more than one digit and that have non-digits in between (use generic classes)

Inline Modifiers, Bounding and Alternation

Inline Modifiers, Bounding and Alternation Topics

- Inline modifiers
- Bounding syntax
- Alternation syntax
- Exercise: Inline modifiers, bounding and alternation
- Exercise solution and Q & A

Inline Modifiers

- PCRE

(`?x`) – embed whitespace

(`?i`) – case insensitive match

(`?s`) – single line mode (`.` matches `\n`)

(`?m`) – multi-line mode (`^` `$` match begin/end of line)

`\A` – beginning of string

`\Z` – end of the string

- Other dialects support other inline modifiers

Inline Modifiers Example

```
$ grep -P '(?xi) ^ [a-z]+ $' words.txt
zERo
One
Two
tHREE
four
FIVE
six
Seven
eight
nine
$ grep -P '(?x) ^ [[:alpha:]]+ $' words.txt
zERo
One
Two
tHREE
four
FIVE
six
Seven
eight
nine
```

Inline Modifiers Examples

- `/(?xs) ^ hello .* world $ /`
 - match a string starting with "hello" and ending with "world", even if that string has `\n` characters (multiple lines as a single string)

Inline Modifiers Examples

- `/(?xs) ^ hello .* world $ /`
 - match a string starting with "hello" and ending with "world", even if that string has `\n` characters (multiple lines as a single string)
- `/(?xm) ^ world /`
 - match a string that has a line that begins with "world" (eg. "hello\nworld\n")

Inline Modifiers Example

```
$ echo 'hello  
> world' | grep -Pz '(?xs) ^ hello .* world'  
hello  
world  
$ echo 'hello  
world' | grep -Pz '(?xm) ^ world'  
hello  
world
```

Bounding

- We have seen two bounding characters

^ – beginning of the string
\$ – end of the string (or right before \n at the end)

- Alternatives (useful when using (?m)):

\A – beginning of string
\Z – end of string (or right before \n at the end)

- Word boundaries:

\b – beginning or end of a word
\B – not the beginning or end of a word

Bounding Examples

```
$ cat text.txt
a line with the in it
a line with other in it
a line with there in it
a line with lathe in it
$ grep -P '\bthe' text.txt
a line with the in it
a line with there in it
$ grep -P 'the\b' text.txt
a line with the in it
a line with lathe in it
$ grep -P '\bthe\b' text.txt
a line with the in it
$ grep -P '\Bthe' text.txt
a line with other in it
a line with lathe in it
$ grep -P 'the\B' text.txt
a line with other in it
a line with there in it
$ grep -P '\Bthe\B' text.txt
a line with other in it
$
```

Alternation

- Match either:

`/a|b/` – either "a" or "b"

Alternation

- Match either:

`/a|b/` – either "a" or "b"
`/one|two/` – either "one" or "two"

Alternation

- Match either:

`/a|b/` – either "a" or "b"
`/one|two/` – either "one" or "two"

- Use parens to apply precedence:

`/in|outside/` – either "in" or "outside"

Alternation

- Match either:

`/a|b/` – either "a" or "b"
`/one|two/` – either "one" or "two"

- Use parens to apply precedence:

`/in|outside/` – either "in" or "outside"
`/(in|out)side/` – either "inside" or "outside"

Alternation

- Match either:

`/a|b/` – either "a" or "b"
`/one|two/` – either "one" or "two"

- Use parens to apply precedence:

`/in|outside/` – either "in" or "outside"
`/(in|out)side/` – either "inside" or "outside"

`/today is (mon|tues)day/` – either "today is monday" or
"today is tuesday"

Alternation

- Low precedence:

`/^first|second|third$/` – "first" at beginning of string, or "second" anywhere, or "third" at the end of the string

`/^(first|second|third)$/` – begin the string, followed by either "first" or "second" or "third", followed by the end of the string

Alternation

- Can match unexpected part of the string:

`"it is outside or inside"`

`/(in|out)side/`

Alternation

- Can match unexpected part of the string:

"it is **outside** or inside"

`/(in|out)side/`

Alternation

- Can match unexpected part of the string:

`"it is outside or inside"`

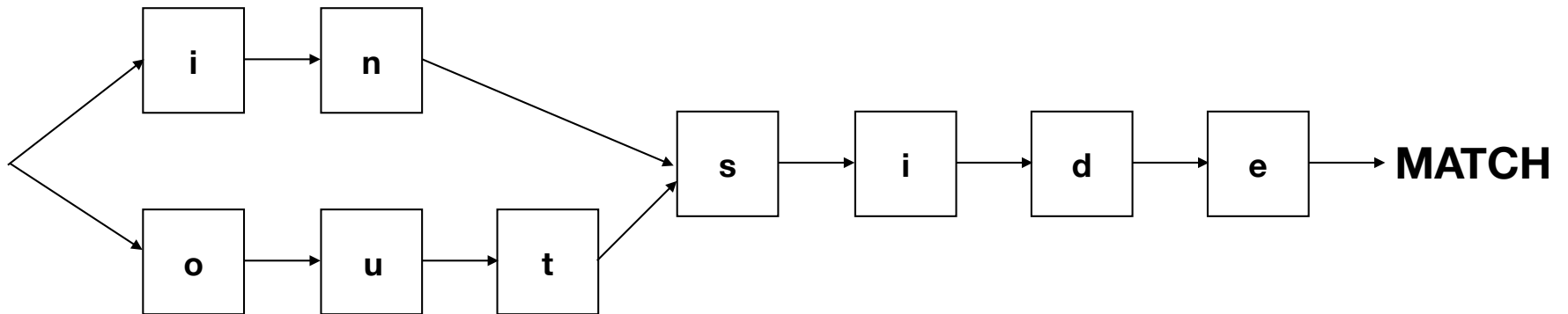
`/(in|out)side/`

- Rule #1 - the earliest match wins

Alternation

"it is outside or inside"

`/(in|out)side/`



Alternation

- Can match unexpected part of the string:

`"two is greater than one"`

`/one|two|three|four/`

Alternation

- Can match unexpected part of the string:

"two is greater than one"

/one|two|three|four/

Alternation Examples

```
$ cat text.txt
this line contains one and two
this line contains two and one
this line contains two and twelve
this line contains twelve and two
first, this line has first in it
second, this line has second in it
third, this line has third in it
$ perl -pe 's/one|two/---/' text.txt
this line contains --- and two
this line contains --- and one
this line contains --- and twelve
this line contains twelve and ---
first, this line has first in it
second, this line has second in it
third, this line has third in it
$ perl -pe 's/two|one/---/' text.txt
this line contains --- and two
this line contains --- and one
this line contains --- and twelve
this line contains twelve and ---
first, this line has first in it
second, this line has second in it
third, this line has third in it
```

Alternation Examples

```
$ perl -pe 's/tw|two/==/' text.txt
this line contains one and ==o
this line contains ==o and one
this line contains ==o and twelve
this line contains ==elve and two
first, this line has first in it
second, this line has second in it
third, this line has third in it
$ perl -pe 's/two|tw/==/' text.txt
this line contains one and ==
this line contains == and one
this line contains == and twelve
this line contains ==elve and two
first, this line has first in it
second, this line has second in it
third, this line has third in it
```

Alternation Examples

```
$ cat text.txt
this line contains one and two
this line contains two and one
this line contains two and twelve
this line contains twelve and two
first, this line has first in it
second, this line has second in it
third, this line has third in it
$ grep -P '^first|second|third' text.txt
first, this line has first in it
second, this line has second in it
third, this line has third in it
$ grep -P '^(first|second|third)' text.txt
first, this line has first in it
second, this line has second in it
third, this line has third in it
```

Exercise 4

- Create regexes to match the following, using inline modifiers as much possible (use the `x` modifier in each regex):
 1. lines that contain "hello" in upper or lower case
 2. words that begin with 2 upper case letters
 3. words that end with either "u" or "z"
 4. words that begin and end with 2 vowels
 5. lines that have more than one occurrence of the word "the"
 6. lines that have more than one occurrence of the word "the" or "there"
 7. lines that begin and end with the word "the"
 8. lines that begin and end with the word "the" or "there" in upper or lower case (do not use POSIX character class)

Q & A



Capturing

Capturing Topics

- Capturing syntax (text extraction) - () and \1
- Matching recurring text
- Replacing text
- Turn off capture with (? :)
- Exercise: Capturing
- Exercise solution and Q & A

Capturing

- Capturing with parens
 - First set of parens stored in \1
 - Second set of parens stored in \2
 - Third set of parens stored in \3
 - Etc
- Can be expensive (have to store text)

Repeated Text

`/(.)\1/` – match two of the same character

`"aa", "bb", "77", "++"`

Repeated Text

`/(.)\1/` – match two of the same character

"aa", "bb", "77", "++"

Repeated Text

`/(.)\1/` – match two of the same character

"aa", "bb", "77", "++"

`/(.)(.)\1\2/` – match two characters twice in a row

"abab", "xyxy", "7878", "+-+-"

`/(.+)\1/` – same

Repeated Text

`/(\.)\1/` – match two of the same character

"aa", "bb", "77", "++"

`/(\.)(\.)\1\2/` – match two characters twice in a row

"abab", "xyxy", "7878", "+-+-"

`/(\.)\1/` – same

Repeated Text

`/(\.)\1/` – match two of the same character

"aa", "bb", "77", "++"

`/(\.)(\.)\1\2/` – match two characters twice in a row

"abab", "xyxy", "7878", "+--+-"

`/(\.)\1/` – same

`/(\.)(\.)\2\1/` – match two chars, reverse them

"abba", "xyyx", "7887", "+---+"

Repeated Text

`/(\.)\1/` – match two of the same character

"aa", "bb", "77", "++"

`/(\.)(\.)\1\2/` – match two characters twice in a row

"abab", "xyxy", "7878", "+-+-"

`/(\.)\1/` – same

`/(\.)(\.)\2\1/` – match two chars, reverse them

"abba", "xyyx", "7887", "+--+"

Repeated Text

```
/ (?x) \b(\w+)\b .* \b\1\b/
```

- a string with a repeated word

Repeated Text

`/(?x) \b(\w+)\b .* \b\1\b/`

- a string with a repeated word

`/(?x) \b (\w+) \s \1\b/`

- the same word twice ("the the ")

Capturing Examples

```
$ cat text.txt
hello
world
vivid
Mississippi
this is a sentence with this twice
here is a sentence with two here words
this is a sentence with no repeats
this has the the two of the same word in a row
$ grep -P '(.)\1' text.txt
hello
world
Mississippi
$ grep -P '(.)(.)\1\2' text.txt
vivid
$ grep -P '(.)(.)\2\1' text.txt
Mississippi
$
```

Capturing Examples

```
$ grep -P '\b(\w+)\b.*\b\1\b' text.txt
this is a sentence with this twice
here is a sentence with two here words
this has the the two of the same word in a row
$ grep -P '\b(\w+)\s\1\b' text.txt
this has the the two of the same word in a row
$
$
```

Replacing Text

- Parens and memory are often used to replace text
- To illustrate, we need to briefly introduce sed (we will talk more about sed, the Stream Editor, later (-E uses ERE)):

```
$ echo 'tonight' | sed -E 's/night/day/'  
today
```

Replacing Text

- Parens and memory are often used to replace text
- To illustrate, we need to briefly introduce sed (we will talk more about sed, the Stream Editor, later (-E uses ERE)):

```
$ echo 'tonight' | sed -E 's/night/day/'  
today
```

```
$ echo '123' | sed -E 's/[0-9]/z/'  
z23
```

Replacing Text

- Parens and memory are often used to replace text
- To illustrate, we need to briefly introduce sed (we will talk more about sed, the Stream Editor, later (-E uses ERE)):

```
$ echo 'tonight' | sed -E 's/night/day/'  
today
```

```
$ echo '123' | sed -E 's/[0-9]/z/'  
z23
```

```
$ echo '123' | sed -E 's/[0-9]/z/g'  
zzz
```

Replacing Text

```
$ echo '123' | sed 's/\(.\) \(.\) /\2\1/'  
213
```

Replacing Text

```
$ echo '123' | sed 's/\(.\) \(.\) /\2\1/'  
213
```

```
$ echo '123' | sed -E 's/(\.)(\.) /\2\1/'  
213
```

Replacing Text

```
$ echo '123' | sed 's/\(.\) \(.\) /\2\1/'  
213
```

```
$ echo '123' | sed -E 's/(\.)(\.) /\2\1/'  
213
```

```
$ echo 'the the' | sed -E 's/(\w+)\s\1\b /\1/'  
the
```


Turn Off Capture

- To turn off capture, use (?:):

```
s/(one|two) (\w+)/\2/
```

Turn Off Capture

- To turn off capture, use (?:):

```
s/(one|two) (\w+)/\2/
```

```
s/(?:one|two) (\w+)/\1/
```

Turn Off Capture

- To turn off capture, use (?:):

```
s/(one|two) (\w+)/\2/
```

```
s/(?:one|two) (\w+)/\1/
```

```
/(?x) (?: https? | ftp) :\/\/ ([^/]+) (/.*)? /  
  \1 is 'www.example.com'  
  \2 is '/test/one/two.html'
```

Turn Off Capture

- To turn off capture, use (?:):

```
s/(one|two) (\w+)/\2/
```

```
s/(?:one|two) (\w+)/\1/
```

```
/(?x) (?: https? | ftp) :\\\/ ( [^/ ]+ ) ( /. * )? /
```

```
\1 is 'www.example.com'
```

```
\2 is '/test/one/two.html'
```

Turn Off Capture

- To turn off capture, use (?:):

```
s/(one|two) (\w+)/\2/
```

```
s/(?:one|two) (\w+)/\1/
```

```
/(?x) (?: https? | ftp) :\/\/ ( [^/ ]+ ) ( /.*)? /  
  \1 is 'www.example.com'  
  \2 is '/test/one/two.html'
```

- We will see later that the captured content can be used after the regex has completed execution

Exercise 5

1. Match a string that starts with an upper alpha and has that character later in the string (example: "My name is Mark")
2. Match a string that has two occurrences of a word where that words ends the string.
3. The Acme Corporation is changing its name to Acme Coyote. Write a regular expression to change all occurrences of "Acme" to "Acme Coyote" and "acme" to "acme Coyote" in a document.
4. Some editors say there should only be 1 space character after a sentence. Write a regex to substitute 2 or more spaces after a period followed by an upper alpha with a single space.

Lazy Quantifiers

Lazy Quantifiers Topics

- Lazy quantifier syntax
- Exercise: Lazy Quantifiers
- Exercise Solution and Q & A

Lazy Quantifier Syntax

- To match minimally, add a ? after the quantifier:

*? – zero or more minimal

ab*?c – "a", zero or more "b" (minimal), "c"

+? – one or more minimal

ab+?c – "a", one or more "b" (minimal), "c"

?? – zero or one minimal

ab??c – "a", zero or one "b" (minimal), "c"

Lazy Quantifier Syntax

- To match minimally, add a ? after the quantifier:

*? – zero or more minimal

ab*?c – "a", zero or more "b" (minimal), "c"


+? – one or more minimal

ab+?c – "a", one or more "b" (minimal), "c"

?? – zero or one minimal

ab??c – "a", zero or one "b" (minimal), "c"

Lazy Quantifier Syntax

- To match minimally, add a ? after the quantifier:

*? – zero or more minimal

ab*?c – "a", zero or more "b" (minimal), "c"


+? – one or more minimal

ab+?c – "a", one or more "b" (minimal), "c"


?? – zero or one minimal

ab??c – "a", zero or one "b" (minimal), "c"

Lazy Quantifier Syntax

- To match minimally, add a ? after the quantifier:

*? – zero or more minimal

ab*?c – "a", zero or more "b" (minimal), "c"


+? – one or more minimal

ab+?c – "a", one or more "b" (minimal), "c"


?? – zero or one minimal

ab??c – "a", zero or one "b" (minimal), "c"


Lazy Quantifier Syntax

$\{n\}?$ – n times minimal

$ab\{3\}?c$ – "a", 3 "b" (minimal), "c"

$\{n,\}?$ – n or more minimal

$ab\{3,\}?c$ – "a", three or more "b" (minimal), "c"

$\{n,m\}?$ – n through m minimal

$ab\{3,5\}?c$ – "a", 3, 4 or 5 "b" (minimal), "c"

Lazy Quantifier Syntax

$\{n\}?$ - n times minimal

$ab\{3\}c$ - "a", 3 "b" (minimal), "c"


$\{n, \}$? - n or more minimal

$ab\{3, \}c$ - "a", three or more "b" (minimal), "c"

$\{n, m\}?$ - n through m minimal

$ab\{3, 5\}c$ - "a", 3, 4 or 5 "b" (minimal), "c"

Lazy Quantifier Syntax

$\{n\}?$ - n times minimal

$ab\{3\}c$ - "a", 3 "b" (minimal), "c"


$\{n, \}$? - n or more minimal

$ab\{3, \}c$ - "a", three or more "b" (minimal), "c"


$\{n, m\}?$ - n through m minimal

$ab\{3, 5\}c$ - "a", 3, 4 or 5 "b" (minimal), "c"

Lazy Quantifier Syntax

$\{n\}?$ - n times minimal

$ab\{3\}c$ - "a", 3 "b" (minimal), "c"


$\{n,\}$? - n or more minimal

$ab\{3,\}c$ - "a", three or more "b" (minimal), "c"


$\{n,m\}?$ - n through m minimal

$ab\{3,5\}c$ - "a", 3, 4 or 5 "b" (minimal), "c"


Lazy Quantifier Examples

```
$ tail -20 macbeth-acti.txt | grep -P '(?xi) a.*d '
For thy undaunted mettle should compose
Nothing but males. Will it not be received,
When we have mark'd with blood those sleepy two
Of his own chamber and used their very daggers,
That they have done't?
LADY MACBETH
As we shall make our griefs and clamour roar
I am settled, and bend up
Away, and mock the time with fairest show:
False face must hide what the false heart doth know.
$ tail -20 macbeth-acti.txt | grep -P '(?xi) a.*?d '
For thy undaunted mettle should compose
Nothing but males. Will it not be received,
When we have mark'd with blood those sleepy two
Of his own chamber and used their very daggers,
That they have done't?
LADY MACBETH
As we shall make our griefs and clamour roar
I am settled, and bend up
Away, and mock the time with fairest show:
False face must hide what the false heart doth know.
$
```

Lazy Quantifier Examples

```
$ tail -12 macbeth-acti.txt | grep -P '(?xi) [aeiou] .* [aeiou] '
That they have done't?
LADY MACBETH
Who dares receive it other,
As we shall make our griefs and clamour roar
Upon his death?
MACBETH
I am settled, and bend up
Each corporal agent to this terrible feat.
Away, and mock the time with fairest show:
False face must hide what the false heart doth know.
Exeunt
$ tail -12 macbeth-acti.txt | grep -P '(?xi) [aeiou] .*? [aeiou] '
That they have done't?
LADY MACBETH
Who dares receive it other,
As we shall make our griefs and clamour roar
Upon his death?
MACBETH
I am settled, and bend up
Each corporal agent to this terrible feat.
Away, and mock the time with fairest show:
False face must hide what the false heart doth know.
Exeunt
$ _
```

Exercise 6

1. Match a string with two integers separated by a space. Match all digits of the first integer with a greedy match, but only match the first digit of the second integer with a lazy match.
2. Change the above regex so that both quantifiers are lazy. What happens?
3. Use a lazy quantifier to match the first quoted string (the line contains more than one quote within double quotes).
4. Use a lazy quantifier to match the last quoted string (the line contains more than one quote within double quotes).
5. Match a comma followed later by a comma (using a possessive quantifier).

Q & A



Lookaround Topics

- Lookarounds - (?=), etc
- Exercise: Lookarounds
- Exercise solution and Q & A

Lookarounds

- Zero length assertions (like ^ and \$)
- Match characters, then give them back (do not consume them)
- Only determine match or no match

Lookaheads

- Lookahead

(?=) - positive lookahead - must be next

(?!) - negative lookahead - cannot be next

Lookahead Examples

"abcd"

/ab(?=c) /

Lookahead Examples

"abcd"



/ab(?=c)/



Lookahead Examples

"**a**bcd"



/ab(?=c) /



Lookahead Examples

"abcd"



/ab(?=c)/



Lookahead Examples

"abcd"



/ab(?=c)/



MATCH

("c" is next, and "c" is not consumed)

Lookahead Examples

"abcd"

/ab(?=d)/

Lookahead Examples

"abcd"



/ab(?=d)/



Lookahead Examples

"a**a**bcd"



/ab(?=d) /



Lookahead Examples

"abcd"



/ab(?=d)/



Lookahead Examples

"abcd"
↑

/ab(?=d) /
↑

FAIL

(because the next character is not a "d")

Lookahead Examples

`"275i"`

`/\d+(?=[aeiou])/`

Lookahead Examples

"275i"



/\d+(?=[aeiou])/



Lookahead Examples

"275i"



/\d+(?=[aeiou])/



Lookahead Examples

"275i"



/\d+(?=[aeiou])/



MATCH

("i" is not consumed)

Lookahead Examples

"275x"

/\d+(?=[aeiou])/

Lookahead Examples

"275x"



/\d+(?=[aeiou])/



Lookahead Examples

"275x"



/\d+(?=[aeiou])/



Lookahead Examples

"275x"



/\d+(?=[aeiou])/



FAIL

(the next character is not a vowel)

Lookahead Examples

"abcd%"

/[a-z]{4}(?!\\d)/

Lookahead Examples

"abcd%"



/[a-z]{4}(?!\\d)/



Lookahead Examples

"abcd%"



/[a-z]{4}(?!\\d)/



Lookahead Examples

"abcd%"



/[a-z]{4}(?!\\d)/



MATCH

("%" is a non-digit, not consumed)

Lookahead Examples

"abcd7"

/[a-z]{4}(?!\\d)/

Lookahead Examples

"abcd7"



/[a-z]{4}(?!\\d)/



Lookahead Examples

"abcd7"



/[a-z]{4}(?!\\d)/



Lookahead Examples

"abcd7"



/[a-z]{4}(?!\\d)/



FAIL

7 is not a non-digit

Password Example

- Match a password:
 - 6 to 8 characters
 - alpha or digits only
 - at least one digit
 - at least one upper case alpha

Password Example

- 6 to 8 characters - alpha or digit:

`/(?x) ^ (?= \w{6,8}) /`

Password Example

- At least one digit:

`/(?x) ^ (?= \w{6,8}) (?.* \d) /`

Password Example

- At least one upper alpha:

`/(?x) ^ (?= \w{6,8}) (?= .* \d) (?= .* [A-Z]) /`

Password Example

- Now match 6 to eight word characters:

```
/(?x) ^ (?= \w{6,8}) (?= .* \d) (?= .* [A-Z]) \w{6,8} $ /
```

Password Example

- No need to repeat the 6 to 8 word chars:

```
/(?x) ^ (?= .* \d) (?= .* [A-Z]) \w{6,8} $ /
```

Lookbehinds

- Lookbehind

(?<=) – positive lookbehind - must be before

(?<!) – negative lookbehind - cannot be before

Lookbehind Examples

"abcd"

`/^\w*(?<=b)/`

Lookbehind Examples

"abcd"



/^\w*(?<=b)/



Lookbehind Examples

"abcd"



/^\w*(?<=b)/



Lookbehind Examples

"abcd"



/^\w*(?<=b)/



Lookbehind Examples

"abcd"



/^\w*(?<=b)/



Lookbehind Examples

"abcd"



/^\w*(?<=b)/



Lookbehind Examples

"abcd"



/^\w*(?<=b)/



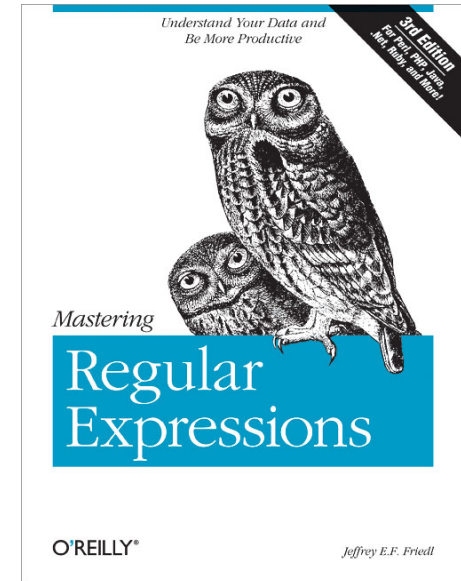
MATCH

Lookaround Example

- Comma-fy an integer:
 - convert 1234567890 to 1,234,567,890
 - first try:

```
s/(\d)(\d\d\d)/\1,\2/g
```

```
1,2345,67890
```



Lookaround Example

- Let's use positive lookbehind:

```
s/(?<=\d)(\d\d\d)/,\1/g
```

```
1,234,567,890
```

```
1,234,567,8901
```

Lookaround Example

- Let's use positive lookbehind:

```
s/(?<=\d)(\d\d\d)/,\1/g
```

```
1,234,567,890
```

```
1,234,567,8901
```

- Need a positive lookahead and negative lookahead:

```
s/(?<=\d)(?=(\d\d\d)+(?!\d))/,/g
```

```
1,234,567,890
```

```
12,345,678,901
```

Exercise 7

1. Using positive lookahead, match the string `"expression"`, but the matched text only includes `"express"`
2. Using positive lookbehind, write a substitution to convert `'{a:test,b:quiz}'` to `'{a:"test",b:"quiz}"`

Q & A



Practical, Efficient and Readable Regular Expressions

Practical, Efficient and Readable Regular Expression Topics

- Practical regular expressions - solving real problems
- Efficient regular expressions - writing faster regexes
- Readable regular expressions - writing "subroutines"
- Exercise: Practical, efficient and readable regular expressions
- Exercise solution and Q & A

Practical Regular Expressions

- Practical regular expressions - solving real problems
 - international phone number
 - valid date
 - email address

Practical Regular Expressions

- International phone number (ITU-T E.123):

a leading plus sign followed by 6 to 15 digits, ending in a digit, can include spaces

```
/(?x)
^          # begin the string
\+        # a plus character
(?:      # group but don't capture
  \d      # digit
  \x20 ?  # optional space
){5,14}   # repeat group 5 to 14 times
\d        # must end in a digit
$         # end of string
/
```

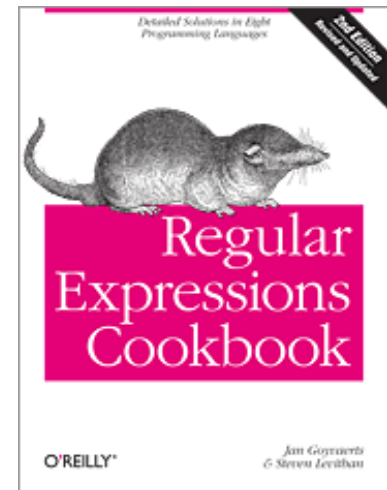


Practical Regular Expressions

- Valid date:

m/d/yy, m/d/yyyy, mm/dd/yy, mm/dd/yyyy,
d/m/yy, d/m/yyyy, dd/mm/yy, dd/mm/yyyy

```
/(?x)
^(?:
  # m/d or mm/dd
  (1[0-2]|0?[1-9])\/(3[01]|[12][0-9]|0?[1-9])
|
  # d/m or dd/mm
  (3[0-1]|[12][0-9]|0?[1-9])\/(1[0-2]|0?[1-9])
)
# /yy or /yyyy
\/(?:[0-9]{2})?[0-9]{2}
$
/
```



Practical Regular Expressions

- Email address:

`user@example.com`

`/(?x) ^ \S+ @ \S+ $ /`

`user-1@example.com`

`/(?xi) ^ [\w.+ -]+ @ [a-z0-9.-]+ $ /`

no leading, trailing or consecutive dots

`/(?xi) ^ [\w+ -]+ (?: \. [\w.+ -]+ )* @ [a-z0-9-]+ (?: \. [a-z0-9-]+ )* $ /`



Practical Regular Expressions

- 99.99% of all email addresses:

```
\A[a-z0-9!#$%&'*/=?^_`{|}~]+(?:\. [a-z0-9!#$%&'*/=?^_`{|}~]+)*@(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.)+[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\Z
```

- RFC 5322:

```
\A(?:[a-z0-9!#$%&'*/=?^_`{|}~]+(?:\. [a-z0-9!#$%&'*/=?^_`{|}~]+)*  
| "(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21\x23-\x5b\x5d-\x7f]  
| \\[\x01-\x09\x0b\x0c\x0e-\x7f])*)" )  
@ (?: (?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.)+[a-z0-9](?:[a-z0-9-]*[a-z0-9])?  
| \[(?: (?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.){3}  
 (?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?|[a-z0-9-]*[a-z0-9]:  
 (?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f]  
 | \\[\x01-\x09\x0b\x0c\x0e-\x7f])+) )  
 \])\Z
```

Efficient Regular Expressions

- Optimize quantifier "loops"
 - `.*` is usually very inefficient - `/" . * "/`

Efficient Regular Expressions

- Optimize quantifier "loops"
 - `.*` is usually very inefficient - `/".*"/`
 - use minimal matching (usually, depends on string)
 - `/".*?"/`

Efficient Regular Expressions

- Optimize quantifier "loops"
 - `.*` is usually very inefficient - `/".*"/`
 - use minimal matching (usually, depends on string)
- `/".*?"/`
 - use mutual exclusion - `/"[^"]*" /`

Efficient Regular Expressions

- Optimize quantifier "loops"
 - `.*` is usually very inefficient - `/".*"/`
 - use minimal matching (usually, depends on string)
- `/".*?"/`
 - use mutual exclusion - `/"[^"]*" /`
 - use possessive (does not keep track) - `/"[^"]"*+ /`

Efficient Regular Expressions

- Which quantifier type do I use?
 - the one that produces the correct result

Efficient Regular Expressions

- Which quantifier type do I use?
 - the one that produces the correct result
 - the one that produces the least amount of backtracking

Efficient Regular Expressions

- Which quantifier type do I use?
 - the one that produces the correct result
 - the one that produces the least amount of backtracking
 - depends on your data

Efficient Regular Expressions

- Which quantifier type do I use?
 - the one that produces the correct result
 - the one that produces the least amount of backtracking
 - depends on your data
 - experiment

Efficient Regular Expressions

- Eliminate backtracking in loops
 - use possessive (does not keep track) - `/ "[^"]*" *+ /`
 - `/ (?x) ^ \w++ (?: , \w++) *+ $ /`

Efficient Regular Expressions

- Eliminate backtracking in alternations
 - quantifiers are not the only things that backtrack
 - refactor for better performance:

```
/ ( ? : abc | abd | abe | abf ) xy /  
/ ab ( ? : c | d | e | f ) xy /
```

- consider using *atomic groups* (aka *possessive groups*)
 - only for mutually exclusive alternatives

Efficient Regular Expressions

- Atomic groups:

`"instinct"`

`/^(inside|integer|instinct)$/`

Efficient Regular Expressions

- Atomic groups:

`"instinct"`
↑
`/^(inside|integer|instinct)$/`
↑

Efficient Regular Expressions

- Atomic groups:

`"i`nstinct"

↑

`/^(inside|integer|instinct)$/`

↑

Efficient Regular Expressions

- Atomic groups:

`"instinct"`
↑

`/^(inside|integer|instinct)$/`
↑

Efficient Regular Expressions

- Atomic groups:

`"instinct"`
 ↑

`/^(inside|integer|instinct)$/`
 ↑

Efficient Regular Expressions

- Atomic groups:

`"instinct"`
↑
`/^(inside|integer|instinct)$/`
↑

Efficient Regular Expressions

- Atomic groups:

`"instinct"`
↑

`/^(inside|instinct)$/`
↑

Efficient Regular Expressions

- Atomic groups:

"**in**stinct"



/^(inside|integer|instinct)\$/



Efficient Regular Expressions

- Atomic groups:

`"instinct"`



`/^(inside|integer|instinct)$/`



Efficient Regular Expressions

- Atomic groups:

`"instinct"`
↑

`/^(inside|integer|instinct)$/`
↑

Efficient Regular Expressions

- Atomic groups:

`"instinct"`
↑

`/^(inside|integer|instinct)$/`
↑

MATCH

after much backtracking

Efficient Regular Expressions

- Atomic groups:

```
"today is tuesday"
```

```
/(?>mon|tues|wednes)day$/
```

```
?> says "no backtracking"
```

```
good for mutually exclusive alternatives
```

```
no backtrack memory
```

Efficient Regular Expressions

- Atomic groups:

"today is tuesday"

/(?>mon|tues|wednes)day\$/

Efficient Regular Expressions

- Atomic groups:

"today is tuesday"



/(?>mon|tues|wednes)day\$/



fast forward...

Efficient Regular Expressions

- Atomic groups:

"today is **t**uesday"

/(?>mon|**t**ues|wednes)day\$/

Efficient Regular Expressions

- Atomic groups:

"today is **tuesday**"



/(?>mon|tues|wednes)day\$/



Efficient Regular Expressions

- Atomic groups:

"today is **tuesday**"

`/(?>mon|tues|wednes)day$/`

Efficient Regular Expressions

- Atomic groups:

"today is tuesday"

/(?>mon|tues|wednes)day\$/

Efficient Regular Expressions

- Atomic groups:

"today is **tuesday**"
↑

/(?>mon|tues|wednes)day\$/
↑

Efficient Regular Expressions

- Atomic groups:

"today is **tuesday**"
↑

/(?>mon|tues|wednes)day\$/
↑

Efficient Regular Expressions

- Atomic groups:

"today is **tuesday**"
↑

/(?>mon|tues|wednes)day\$/
↑

MATCH

efficiently, nothing remembered
for backtracking

Readable Regular Expressions

- Use "subroutines"
 - Defined within the (? (DEFINE) . . .) block
 - Defined with (? <NAME> . . .)
 - "Called" with (? &NAME)
- Full example later using Python

Readable Regular Expressions

```
/(?xmi)
  (? (DEFINE)
    (?<NAME>    \w+  \s  \w+
    (?<PHONE>    \+  (? : \d  \s?  ){5,14}  \d  )
    (?<EMAIL>    [\w.+-]+  @  [a-z0-9.-]+
  )

  ^  Name:  \s  (?&NAME)  $  \s
  ^  Phone: \s  (?&PHONE) $  \s
  ^  Email: \s  (?&EMAIL) $  \s
/
```

Readable Regular Expressions

```
/(?xmi)
```

```
(?(DEFINE)
```

```
    (?<NAME>    \w+ \s \w+                                )
```

```
    (?<PHONE>    \+ (?: \d \s? ){5,14} \d )
```

```
    (?<EMAIL>    [\w.+-]+ @ [a-z0-9.-]+                )
```

```
)
```

```
^ Name:  \s (?&NAME)  $ \s
```

```
^ Phone: \s (?&PHONE) $ \s
```

```
^ Email: \s (?&EMAIL) $ \s
```

```
/
```

Readable Regular Expressions

```
/(?xmi)
  (? (DEFINE)
    (?<NAME> \w+ \s \w+ )
    (?<PHONE> \+ (? : \d \s? ){5,14} \d )
    (?<EMAIL> [\w.+-]+ @ [a-z0-9.-]+ )
  )

  ^ Name: \s (?&NAME) $ \s
  ^ Phone: \s (?&PHONE) $ \s
  ^ Email: \s (?&EMAIL) $ \s
/
```

Exercise 8

1. Write a regex to match all the valid URLs in `urls.txt`, but not any of the invalid URLs.
2. Use `sed` to turn each valid URL into a link.
3. Write a regex to match valid Canadian postal codes. They are alternating upper alpha and digits, for example: "V5K 0V1". The space is optional. There are a few alphas that are not allowed - D, F, I, O, Q, U - use a negative lookahead to eliminate these. Additional invalid alphas for the first position are W and Z.
4. Use `sed` to substitute all dates in the format (`dates.txt`):
12-02-2022
to:
2022-12-02
5. Write an regex with no backtracking to match a string in this format:
<77,3949,29595,113>
6. Optional: rewrite #4 above using "subroutines" (we need to talk about Python to demonstrate this solution).

Q & A



Programming Languages

Programming Languages Topics

- JavaScript
- Python
- If we have time: sed and awk
- Exercise: Programming languages
- Exercise solution and Q & A

JavaScript

- Matching with `match ( )`
- Groups

JavaScript Matching

- `match ( )` matches a regex against a string
- For PCRE, use the XRegExp module:

```
$ npm install xregexp
```

```
var XRegExp = require('xregexp')
```

JavaScript Match Example

```
// match.js

// use str.match() to match a regex
var str = 'hello, world!';
if (str.match(/(.)\1/)) {
    console.log('MATCH 1!');
}

if (str.match(/^w+,s\w+!$/)) {
    console.log('MATCH 2!');
}

// for all of PCRE
// npm install xregexp
var XRegExp = require('xregexp');
var re = XRegExp(
    ^      # begin the string      \
    \\w+   # one or more word chars \
    ,      # comma                  \
    \\s    # a space                \
    \\w+   # one or more word chars \
    !      # exclamation point      \
    $      # end the string         \
    ', 'x');

if (str.match(re)) {
    console.log('MATCH 3!');
}

// can also use re.test to match
if (re.test(str)) {
    console.log('MATCH 4!');
}
```

JavaScript Groups

- The `match()` method returns an object:

```
var result = str.match('someregex');
```

- `result[0]` - part of the string that matched
- `result[1]` - first group
- `result[2]` - second group
- `result.index` - index in string where matched text begins
- `result.input` - entire string

JavaScript Group Example

```
// groups.js

var str = 'regexes are awesome';

var result = str.match(/\w+/g);
console.log(result);
console.log(result[0]);
console.log(result[1]);
console.log(result[2]);

console.log('-----');
var result = str.match(/\s(\w+)\s/);
console.log(result);
console.log(result[0]);
console.log(result[1]);
console.log(result.index);
console.log(result.input);

console.log('-----');
var result = str.match(/^( \w+ ).* \s( \w+ )/);
console.log(result);
console.log(result[0]);
console.log(result[1]);
console.log(result[2]);
```

Python

- Matching with `match ( )` and `search ( )`
- Groups
- "Subroutine" example

Python Matching

- `match( )` only matches at the beginning of the string
- `search( )` does not default to matching at the beginning of the string

```
match( 'hello' )
```

is the same as:

```
search( '^hello' )
```

- `search( )` is recommended over `match( )`

Python Compiling Regex

- For complex regexes, `compile()` them and use the compiled object to call `search()`
- Can use options such as `re.X` and `re.I`
 - Combine with the vertical bar:

`re.X | re.I`

Python Search Example

```
# search.py

import re

s = 'Python uses the re module'

if (re.search('re module', s)):
    print('MATCH 1!')

# re.match() only matches at the
# beginning of the string
# use re.search() instead
if (re.match('re module', s)):
    print('MATCH 2 - NOT!')

# can create a regex object and
# use it to search
rel = re.compile(r'''
    ^
    .*
    (\s\w+)*
    $
    ''', re.X)

if (rel.search(s)):
    print('MATCH 3!')
```

Python Groups

- Text matched within grouping parens are available with the following methods in the match object `m`:
 - `m.groups()` - a list of all matched groups
 - `m.group(0)` - the part of the string that matched the regex
 - `m.group(1)` - the 1st group
 - `m.group(2)` - the 2nd group
 - etc...

Python Group Example

```
# groups.py

import re

s = 'john@example.com'

rel = re.compile(r'''
    ^
    (\w+)      # user
    @
    (\w+)      # left part of domain
    \.
    (\w+)      # right part of domain
    $
''', re.X)

# like //g
m = rel.search(s)

if m:
    print('MATCH!')
    print('match object:', m)
    print('all groups:', m.groups())
    print('group 0:', m.group(0))
    print('group 1:', m.group(1))
    print('group 2:', m.group(2))
    print('loop through the groups:')
    for group in m.groups():
        print('    ', group)
```

Python "Subroutines"

- The following example illustrates writing regex macros, or "subroutines":
 - Defined within the (? (DEFINE) . . .) block
 - Defined with (? <NAME> . . .)
 - Called with (? &NAME)

Python Subroutine Example

```
# subroutines.py

# sudo pip3 install regex
import regex

s = '''
Name: John Doe
Phone: +1 312 555 1212
Email: john@doe.org
'''

rel = regex.compile('''
    (? (DEFINE)
        (?<NAME> \w+ \s \w+ )
        (?<PHONE> \+ (? : \d \s? ){5,14} \d )
        (?<EMAIL> [\w.+-]+ @ [a-z0-9.-]+ )
    )

    ^ Name: \s (?&NAME) $ \s
    ^ Phone: \s (?&PHONE) $ \s
    ^ Email: \s (?&EMAIL) $ \s
''', regex.X | regex.M | regex.I)

m = rel.search(s)
if (m):
    print('MATCH!')
    print(m)
```

If We Have Time: sed and awk

- Two useful text processing tools in Linux that use regular expressions:
 - sed - the **S**tream **E**Ditor
 - awk - named after its creators, **A**ho, **W**einberger and **K**ernigan

sed

- sed edits streams of text
- The dialect for sed is BRE
- sed -E for ERE
- man sed for details

sed Examples

- can do substitution:

```
$ echo 'Today is the day' | sed 's/day/night/'  
Tonight is the day
```

sed Examples

- can do substitution:

```
$ echo 'Today is the day' | sed 's/day/night/'  
Tonight is the day
```

```
$ echo 'Today is the day' | sed 's/day/night/g'  
Tonight is the night
```

sed Examples

- text matched is in &:

```
$ sed 's/[a-z]*/(&)/g'
```

```
one two
```

```
(one) (two)
```

```
three four five
```

```
(three) (four) (five)
```

sed Examples

- text captured in \1, \2, etc:

```
$ sed -E 's/([a-z]*) ([0-9]*)/text: \1, digits: \2/'  
hello 123  
text: hello, digits: 123  
world 456  
text: world, digits: 456  
^D
```

sed Examples

```
$ cat quote.txt
```

```
Hello, world!
```

```
-- Dennis Ritchie
```

```
Brian Kernigan
```

```
$ sed -E 's/(.)\1/+/g' quote.txt
```

```
He+o, world!
```

```
+++++ De+is Ritchie
```

```
+++++ Brian Kernigan
```

sed Examples

remove space and tab at beginning of line

```
sed -E 's/^[ \t]*//' file_with_initial_whitespace.txt
```

sed Examples

remove space and tab at beginning of line

```
sed -E 's/^[ \t]*//'file_with_initial_whitespace.txt
```

remove space and tab at end of line

```
sed -E 's/[ \t]*$//' file_with_trailing_whitespace.txt
```


sed Examples

remove space and tab at beginning of line

```
sed -E 's/^[ \t]*//' file_with_initial_whitespace.txt
```

remove space and tab at end of line

```
sed -E 's/[ \t]*$//' file_with_trailing_whitespace.txt
```

remove space and tab at beginning and end of line

```
sed -E 's/^[ \t]*//;s/[ \t]*$//' file_with_excessive_whitespace.txt
```

sed Examples

```
# add area code to phone numbers
```

```
sed -E 's/^\([0-9][0-9][0-9]-[0-9][0-9][0-9][0-9]\)/ (312) \1/g' phonenumbers.txt
```

sed Examples

```
# add area code to phone numbers
```

```
sed -E 's/^\([0-9][0-9][0-9]-[0-9][0-9][0-9][0-9]\)/ (312) \1/g' phonenumbers.txt
```

```
# more rain today
```

```
sed -E 's/rain/heavy rain/g' seattle-notes.txt
```

awk

- Basic form of an awk program:

awk regex { program_actions }

- Quick example:

```
$ ls -l | awk '/dat/'
```

```
-rw-r--r-- 1 student student 19 Sep 24 15:42 a.dat  
-rw-r--r-- 1 student student 19 Sep 24 15:42 b.dat  
-rw-r--r-- 1 student student 19 Sep 24 15:42 c.dat  
-rw-r--r-- 1 student student 19 Sep 24 15:42 d.dat
```

awk Examples

- Columns are in \$1, \$2, etc:

```
$ ls -l | awk '/dat/ { print $6 }'
```

```
Sep
```

```
Sep
```

```
Sep
```

```
Sep
```

```
$ ls -l | awk '/dat/ { print $9 }'
```

```
a.dat
```

```
b.dat
```

```
c.dat
```

```
d.dat
```

awk Examples

- `-F` is the delimiter (default is white space):

```
$ awk -F: '/^root/ { print $3 }' /etc/passwd  
0
```

awk Examples

- If no pattern is provided, it operates on all lines of text:

```
$ awk -F: '{ print $1, $3 }' /etc/passwd
root 0
daemon 1
bin 2
sys 3
sync 4
games 5
man 6
```

awk Examples

```
# print all usernames in /etc/passwd:
```

```
awk -F: '{print $1}' /etc/passwd
```


awk Examples

```
# print all usernames in /etc/passwd:
```

```
awk -F: '{print $1}' /etc/passwd
```

```
# print username to real name mapping:
```

```
awk -F: '{print $1 "\t==\t" $5}' /etc/passwd
```

Exercise 9

- Use sed to strip white space from beginning and end of each line of the file `ex9-1.txt` using capturing and `\1`.
- Use sed to remove `0x` from the beginning of valid hex numbers in the file `ex9-2.txt`.
- Solve Exercise 8 #4 using subroutines in Python:

Substitute all dates in the formats:

`12-02-2022`

to:

`2022-12-02`

Helpful code is found in `ex9-3.py`.

- Write a Python program to match Roman numerals using readable and efficient techniques. A regex to match unvalidated Roman numerals can be found in `ex9-4.py`.

Q & A



Remember These?

`/(?x) , [ ^ , ] ++ , /`

`/(?<=\.) {2,} (?=[A-Z])/`

`/\(?\d{3}[\ \-.\)]*\d{3}[\ \-.]?\d{4}/`

`/(?x) (? : https? | ftp) :// ([^/]+) (/.*)? /`

`s/(?<=\d)(?=(\d\d\d)+(?!\d))/,/g`

`/(?xi) ^ [\w+-]+ (? : \. [\w.+ -]+ ) * @ [a-z0-9-]+ (? : \. [a-z0-9-]+ ) * $ /`

Thanks for Attending!



Thank
You

james@onsight.com

Exercise Solutions

Exercise 1 Solution

1. `/t/` - two, three, eight
2. `/.n./` - one, nine
3. `/^t/` - two, three
4. `/e$/` - one, three, five, nine
5. `/ne$/` - one, nine
6. `/^...$/` - one, two, six

Exercise 1 Solution

1. `grep -P e words.txt`
2. `grep -P ^e words.txt`
3. `grep -P ^f..e$ words.txt`

Exercise 2 Solution

1. `/[0-9]/` - Ten 10, eleven 11
2. `/[A-Z][a-z]/` - Zero, One, Two, Seven, Ten 10
3. `/\w\s\d/` - Ten 10, eleven 11
4. `/[[:upper:]]/` - zERo, One, Two, tHREE, FIVE, Seven, Ten 10
5. `/^[[:upper:]]/` - One, Two, FIVE, Seven, Ten 10
6. `/[[:^alpha:]]$/` - Ten 10, eleven 11

Exercise 2 Solution

1. `grep -P [A-Z][A-Z][A-Z] words.txt`
2. `grep -P '\d\d\d[ \-.] \d\d\d[ \-.] \d\d\d\d' phonenumbers.txt`

Exercise 3 Solution

1. `/[A-Z]{2}/` - zERo, tHREE, FIVE
2. `/[a-z][A-Z]{2}/` - zERo, tHREE
3. `/([a-z][A-Z]){2}/` - fOuR
4. `/[A-Z]+/` - zERo, One, Two, tHREE, fOuR, FIVE, Seven, Ten 10
5. `/[a-zA-Z]+\s\d+/` - Ten 10, eleven 11
6. `/\w+\s\w+/` - Ten 10, eleven 11
7. `/(?x) \w+ \x20? \w+/` - [all]

Exercise 3 Solution

1. `grep -P '(?x) ^ \d{3} .* \d{3} $' ex3.txt`

2. `grep -P '(?x) ^ \d{3} .* [aeiou]{2} .* [a-z]{3} $' ex3.txt`

3. `grep -P '(?x) ^ \d{2,} \D+ \d{2,} $' ex3.txt`

Exercise 4 Solution

1. `grep -P '(?xi) hello' ex4.txt`
2. `grep -P '(?x) \b[A-Z]{2}' ex4.txt`
3. `grep -P '(?x) [uz]\b' ex4.txt`
4. `grep -P '(?xi) \b[aeiou]{2} \w* [aeiou]{2}\b' ex4.txt`
5. `grep -P '(?x) \bthe\b .* \bthe\b' ex4.txt`
`grep -P '(?x) (\bthe\b .*){2}' ex4.txt`

Exercise 4 Solution

6.

```
grep -P '(?x) \b(the|there)\b .* \b(the|there)\b' ex4.txt
grep -P '(?x) (\b(the|there)\b .*){2}' ex4.txt
grep -P '(?x) \bthe(re)?\b .* \bthe(re)?\b' ex4.txt
grep -P '(?x) (\bthe(re)?\b .*){2}' ex4.txt
```
7.

```
grep -P '(?x) ^ the\b .* \bthe $' ex4.txt
```
8.

```
grep -P '(?xi) ^ the(re)?\b .* \bthe(re)? $' ex4.txt
```

Exercise 5 Solution

1. `/ (?x) ^ ([A-Z]) .* \1/`

2. `/ (?x) \b (\w+) \b .* \b \1 $ /`

3. `s / (?i) (acme) / \1 Coyote /g`

4. `s / \. {2,} ([A-Z]) / . \1 /g`

Exercise 6 Solution

1. `grep -P '(?x) \d+ \x20 \d+?' ex6.txt`

2. `grep -P '(?x) \d+? \x20 \d+?' ex6.txt`

3. `grep -P '(?x) "[^"]"+?' ex6.txt`

4. `grep -P '(?x) .* "[^"]"+?' ex6.txt`

5. `grep -P '(?x) , [^,]*+ , ' ex6.txt`

Exercise 7 Solution

1. `/express(?=ion)/`

2. `s/(?<=:)\w+/"$&"/g`

Exercise 8 Solution

1.

```
grep -P '(?xi) ^ (https?|ftp|file) :// \S+ $' urls.txt
grep -P '(?xi) ^ (https?|ftp|file) :// [-A-Z0-9+&@#/%?
=~_|$!:,.;]* [A-Z0-9+&@#/%=~_|$]' urls.txt
grep -P '(?xi) ^ ( (https?|ftp|file):// | (www|ftp) \. )
[-A-Z0-9+&@#/%?=~_|$!:,.;]* [A-Z0-9+&@#/%=~_|$]' urls.txt
```
2.

```
sed -E 's/^\((https?|ftp|file):\//\| (www|ftp)\. ) [-A-
Z0-9+&@#/%?=~_|$!:,.;]* [A-Z0-9+&@#/%=~_|$] /<a
href="&">&<\html>/i' urls.txt
```
3.

```
grep -P '(?xi) ^ (?! .* [DFIOQU]) [A-VXY] \d [A-Z] \x20?
\d [A-Z] \d $' postalcodes.txt
```

Exercise 8 Solution

4. `sed -E 's/^\([0-9][0-9]\)-([0-9][0-9])-([0-9][0-9][0-9][0-9])$/\3-\1-\2/g' dates.txt`

5. `/(?x) ^ < \d++ (??:\d++)++ > $ /`

6. `s/(?x)`

`(?(DEFINE)`

`(?<TWO DIGITS> \d\d )`

`(?<MONTH> (?&TWO DIGITS) )`

`(?<DAY> (?&TWO DIGITS) )`

`(?<YEAR> \d\d\d\d )`

`)`

`^ ((?&MONTH)) - ((?&DAY)) - ((?&YEAR)) $`
`/\3-\2-\1/g`

Exercise 9 Solution

1. `sed -E 's/^\s*(.*)\s*$/\1/' < ex9-1.txt`

2. `sed -E 's/0x([a-f0-9]+)$/\1/i' < ex9-2.txt`

Exercise 9 Solution

```
# ex9-3-solution.py

import regex

dates = [
    '02-02-2022',
    '02-22-2022',
    '12-02-2022',
    '12-22-2022'
]

rel = regex.compile('''
    (? (DEFINE)
        (?<TWODIGITS>  \d\d      )
        (?<MONTH>      (?&TWODIGITS) )
        (?<DAY>        (?&TWODIGITS) )
        (?<YEAR>       \d\d\d\d   )
    )

    ^ ((?&MONTH)) - ((?&DAY)) - ((?&YEAR)) $
''', regex.X)

for date in dates:
    m = rel.search(date)
    if (m):
        print('MATCH!')
        print(m)
```

Exercise 9 Solution

```
# ex9-4-solution.py

import regex

numerals = [
    'MCM',
    'LXXX',
    'XVII',
    'MMXIX',
    'III',
    'xlvii'
]

re1 = regex.compile('''
    (? (DEFINE)
        (?<MUSTHAVE> (?=[MDCLXVI]) ) # one of these roman numerals must exist
        (?<MS>          M*           ) # zero or more Ms
        (?<CDSECTION> (? :
            C [MD] |           # C then either M or D
            D? C{0,3}         # optional D and up to 3 C
        ) )
''')
```

Exercise 9 Solution

```
(?<XLSECTION> (?:  
    X [CL] |      # X then either C or L  
    L? X{0,3} )   # optional L and up to 3 X  
)  
(?<IVSECTION> (?:  
    I [XV] |      # I then either X or V  
    V? I{0,3} )   # optional V and up to 3 I  
)  
)  
^  
(?&MUSTHAVE)  
(?&MS)  
(?&CDSECTION)  
(?&XLSECTION)  
(?&IVSECTION)  
$  
'', regex.X | regex.I)
```

```
for numeral in numerals:  
    m = re1.search(numeral)  
    if (m):  
        print(numeral + ' MATCH!')
```